

17-214/514: Agentic Software Development

Design for Change

Hammad Ahmad and Claire Le Goues • Wednesday, September 16, 2026 • **Lecture**

Administrivia

- Midterm example questions will be posted later this week.
- A3, A5 and A6 will build on your A2 repo. No new repo invite needed.
 - A4 will have its own repo. We'll invite you to it when the assignment is released.

Getting unstuck

- **The jargon.** APIs, auth, HTTP. Ask your agent to explain the term at a level you are comfortable with (including "ELI5").
 - Assignment handouts often define new concepts briefly. The A2 contract, for example, has a "New to HTTP APIs?" section.
- **The blank page.** You do not have to invent the plan alone. Ask the agent for a draft. Drop anything the contract does not ask for. Split the rest into agent-sized tasks, each with a check that can fail.
- **The volume of code to read.** You are not expected to read every line the agent writes. Read the structure against your design, the interfaces, the checks, and what each test claims.
- **The number of hours spent already.** If you have been staring at a screen for an hour without progress, *please* come to office hours. Or email us.

What did we talk about last time?

- A design in your head is lost with the context. Write it down.
- Decision records capture the **why**. The decision, the rejected options, the tradeoff. Supersede, never rewrite.
- Lightweight notation is for understanding. Learn to read it.
- Design artifacts double as agent context. Instruction files are the part the agent reads every session.
- Patterns are shared vocabulary, for humans and agents.
- Today: the property all of that is in service of. Can the system **change**?

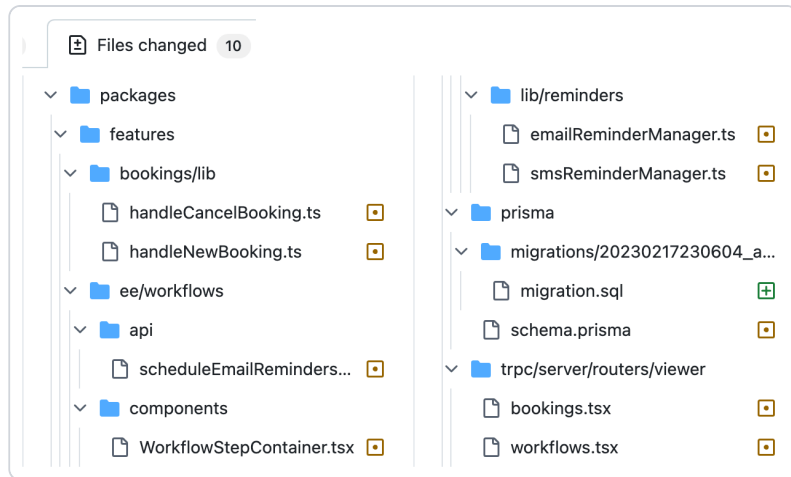
Learning goals for today

- Evaluate a design using coupling and cohesion
- Apply responsibility-assignment heuristics to decide where behavior lives
- Explain how design decisions keep future change cheap
- Explain why modularity is the unit of safe delegation to an agent

One sentence in, ten files out

The request: **"stop sending reminders for bookings that were cancelled."**

- One sentence. The fix touches ten files, across four packages.
- Is that the author's fault?
 - Open the repo. Reminder logic lives in five places. The diff shows the design.
 - The booking handlers, the workflow library, the email and SMS managers, the database schema, two API routers.
- Today: what makes a one-sentence change a ten-file change?



<https://github.com/calcom/cal.com/pull/7232/files>

Designing for change

Why design for change?

- Requirements shift, dependencies move, the business pivots.
- The question is what a change will cost, not whether one is coming.
- A design's job is to make the **likely** changes cheap.
- You likely will not guess the exact future requirement. You can design so that when it arrives, the change stays local.

What a change costs in low-quality code

- A study of 39 production codebases, about 30,000 files, with their issue trackers.
- Low-quality code carried 15 times the defects of high-quality code.
- Resolving an issue in low-quality code took 124 percent longer on average, and the worst cases ran 9 times longer.
- So the design you ship is mostly a bet on how cheap the next changes will be.

Code Red: The Business Impact of Code Quality – A Quantitative Study of 39 Proprietary Production Codebases

Adam Tornhill
CodeScene
Malmö, Sweden
adam.tornhill@codescene.com

Markus Borg
RISE Research Institutes of Sweden
Lund University
Lund, Sweden
markus.borg@ri.se

ABSTRACT

Code quality remains an abstract concept that fails to get traction at the business level. Consequently, software companies keep trading code quality for time-to-market and new features. The resulting technical debt is estimated to waste up to 42% of developers' time. At the same time, there is a global shortage of software developers, meaning that developer productivity is key to software businesses. Our overall mission is to make code quality a business concern, not just a technical aspect. Our first goal is to understand how code quality impacts 1) the number of reported defects, 2) the time to resolve issues, and 3) the predictability of resolving issues on time. We analyze 39 proprietary production codebases from a variety of domains using the CodeScene tool based on a combination of source code analysis, version-control mining, and issue information from Jira. By analyzing activity in 30,737 files, we find that low

of society continues [11, 13, 29, 43, 48]. At the same time as the software industry is struggling with recruiting enough talent, research indicates that up to 42% of developers' time is wasted dealing with Technical Debt (TD) where the cost of subpar code alone comes to \$85 billion annually [57]. This implies that there is an untapped potential in software projects if the code quality is improved and TD paid down.

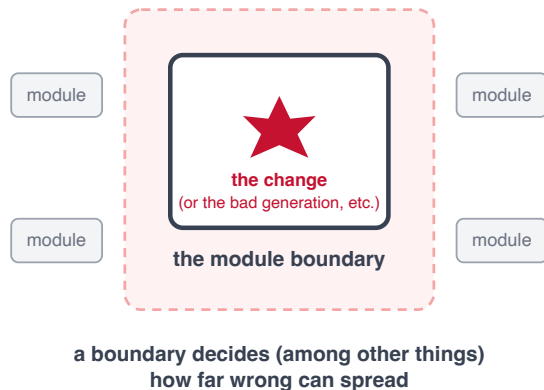
Unfortunately, the software industry often moves in the opposite direction: research finds that developers are frequently forced to introduce new TD [10] as companies keep trading code quality for time to market and new features [24]. This is a decision-making bias known as *hyperbolic discounting*, i.e., humans make choices today that their future selves would prefer not to have made [35].

One reason for the hyperbolic discounting of code quality is that the business impact of code quality remains vague [32]. This was

Tornhill and Borg, "Code Red: The Business Impact of Code Quality," TechDebt (ICSE 2022).

The cost of a change

- A change inside one cohesive module is small.
- The same change smeared across coupled modules is approaching "project" territory.
- Coupling can turn a small requirement into a (big) project.



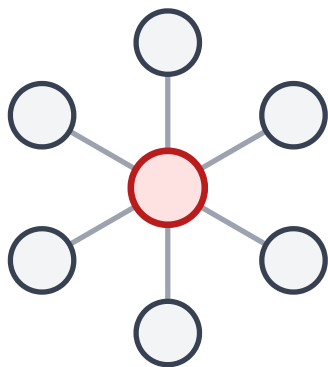
Where the booking system is headed

- Cancellations. Waitlists (today's feature). Recurring bookings. Permissions. Pricing tiers.
- None of this is unheard of. This is what it looks like when software is in use and needs evolve.
- The design question: which of these does the current shape make cheap, and which one does it fight?

Coupling and cohesion

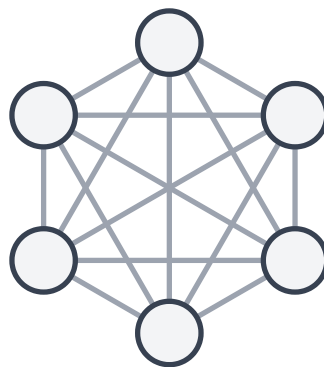
Visualizing coupling

Coupling is how much modules depend on each other, which the dependency graph makes visible. Three module interconnection structures:



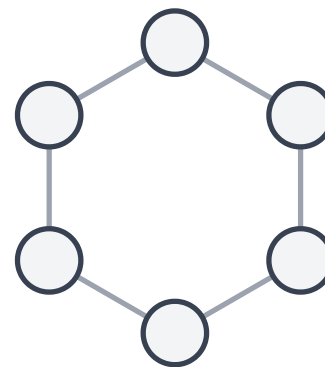
A. One hub

six edges, all via the hub



B. Everyone to everyone

fifteen edges



C. Neighbors only

six edges, nothing central

Tight coupling

```
// tightly coupled: book reaches into everyone's internals
void book(User u, Room r, Slot s) {
  // knows Room's internals
  if (r.bookings.stream().noneMatch(b -> b.overlaps(s))) {
    // knows the store's guts
    r.store.rows.add(new Object[]{u.id, s});
  }
}
```

Change how `Room` stores bookings, or how the store works, and this breaks. It depends on the internals of both.

Loosen it

```
void book(User u, Room r, Slot s) {  
    if (r.isFree(s)) {                // talk through interfaces, not internals  
        r.addBooking(new Booking(u, s));  
    }  
}
```

- Now `Room` can change its internals freely. This code only depends on its **promises**.
- Fewer things we touch, fewer things we break.

What should callers depend on?

- The old maxim: program to an interface, not an implementation.
- Depend on **what** a thing does, not **how** it does it.
- Swap an in-memory store for a real database without touching a caller.
 - Where have we seen this before?
- The interface is the boundary (and the safe place to change, test, and delegate).

Recall: Week 2's doubles needed a boundary

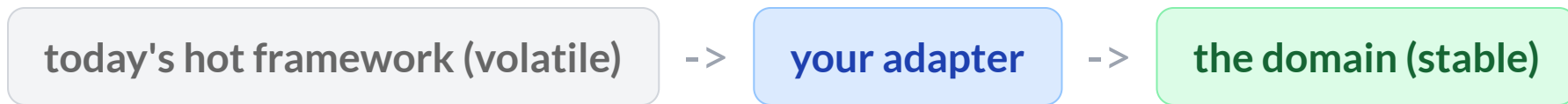
- The recording notifier we swapped in for the real one went in at an interface, `Notifier`.
- Tight coupling gives a double nowhere to go. You end up testing the mail server too.
 - A1's server opened its SQLite file at a hardcoded path, on import. Your tests had nowhere to hand it a different one unless you made a place.
- Testable code is generally easier to change as well.

Hide what's likely to change

- **Encapsulation:** A module keeps its data and its decisions behind an interface. Outside code sees the promise, not the guts.
- Its point is containment (secrecy is a side effect).
- **Information hiding:** The design question that goes with it. Which decision is this module hiding?
- Put the volatile decision (a storage format, an algorithm, a vendor SDK) behind an interface.
 - Callers depend on the promise, so the guts can change without a ripple.

Depend in the direction of stability

- Stable things (a core domain concept, a well-chosen interface) are safe to depend on.
- Volatile things (a vendor SDK, today's hot framework) should depend on **you**, not the other way around.
- Point your dependencies at whatever changes least. (The D in SOLID: dependency inversion.)



Arrows mean "depends on". They all point toward what changes least.

As an aside: this week's lab also operates at a boundary

- Lab 4's template takes an image and a port from you (plus one knob the lab uses to break it).
- The service reads one environment variable, `PORT`. Nothing in it knows that EC2 exists.
- The same container runs on your laptop and on AWS. The lab's broken scenario is a configuration fix, not a code change.

Coupling comes in flavors

- **Data coupling:** You pass only what's needed.
 - `overlaps(slotA, slotB)` : two values in, a boolean out. The whole dependency is the signature.
- **Control coupling:** Flags that tell another module **how** to behave.
 - `process(data, true, 2)` : true **what?** 2 **what?**
- **Hidden coupling:** Shared globals and singletons. The modules never call each other but are still wired together.
- **Temporal coupling:** "Must call `init()` before `use()` ." An unwritten ordering rule.

The further down the list, the harder it is to see.

Coupling the signature doesn't show

```
// pricing/Rates.java
class Rates { static Map<String, Integer> perHour = new HashMap<>(); }

// booking/Quote.java    (never imports admin)
int quote(Room r, int hours) {
    return Rates.perHour.getDefault(r.id, 40) * hours;
}

// admin/Reset.java      (never imports booking)
void resetRates() { Rates.perHour.clear(); }
```

- Neither file imports the other. The import block is clean.
- The booking tests pass on their own. Run them after an admin test and the quote changes.
- Nothing in either signature says so. The dependency is a shared static map.

Temporal coupling

```
BookingService svc = new BookingService();  
svc.loadRooms(catalog);           // forget this line and book() throws  
Booking b = svc.book(user, room, slot);
```

- The ordering rule lives only in the caller's memory. Nothing in the types says "rooms first."
- Every new caller can get it wrong, including an agent writing its first call site.
- The fix makes the wrong order impossible to write. Require the rooms in the constructor, or have the loading step return the service.

```
BookingService svc = new BookingService(catalog); // no wrong order exists
```

The imports don't lie

- The import block can give you an idea of the extent of coupling.
- Ask yourself: who knows whom?
 - A domain class importing the web framework is upside down.

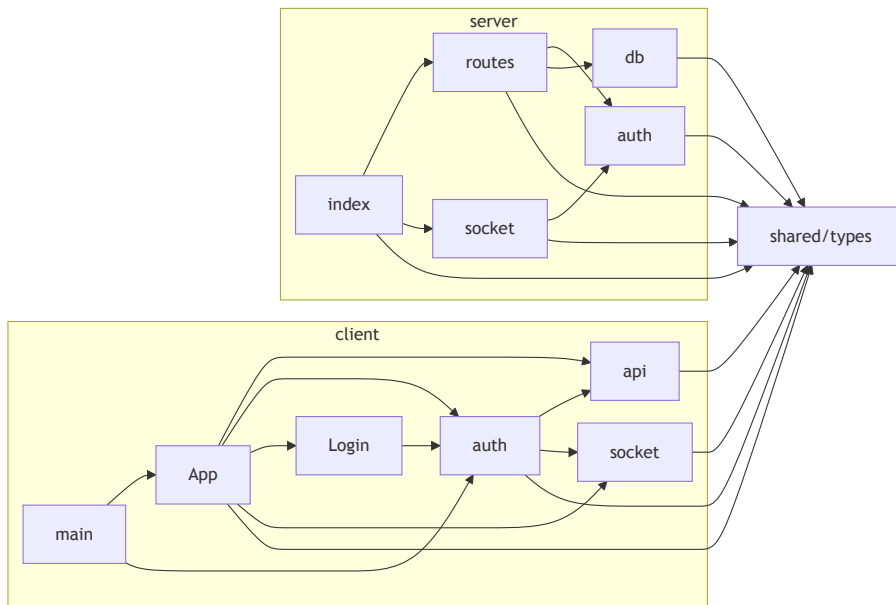
```
// domain/Room.java
import domain.Booking;
import domain.TimeRange;
import java.util.List;

import web.http.RequestContext;
```

Line 6: why does the **domain** know about the **web**?

The import graph of a system you know

- Assignment 1's starter (drawn from its `import` lines). Every arrow is a way a change can travel.
- Two islands. Only `shared/types` joins them.
- A cheap(ish) activity: when planning a big change, have the agent draw this with a tool. Then walk the arrows yourself from the file you will touch (to see the **reach** of the change).

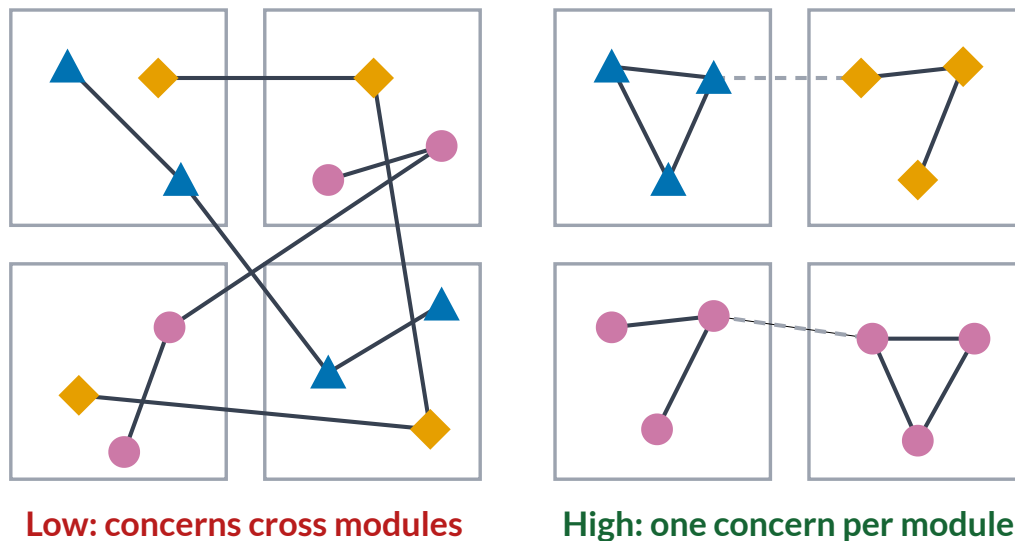


Cohesion

- Last week we asked "one responsibility?" at every zoom level. This is cohesion (and the S in **S**OLID: single responsibility principle).
- **High cohesion:** Everything in a module belongs together and serves one purpose.
- **Low cohesion:** A `Utils` or `Manager` class that does booking, billing, and the weather.
- A low-cohesion module changes for many unrelated reasons.
- Low cohesion also usually results in a high representational gap.
 - (The reverse does not hold, since a module can be cohesive around a concept the domain never had.)

Visualizing cohesion

Cohesion is what a module's insides have in common.



Each shape is a concern, each point a method, each square a module.

Cohesion comes in flavors too

- **Functional:** Everything in the module serves one job. `Room` holds rooms and answers questions about rooms.
- **Sequential:** Steps that feed each other. Parse, validate, store, kept together because they always run in that order.
- **Coincidental:** Things that share a file and nothing else. The `Manager` class.

A low-cohesion class

```
class BookingManager {  
    void book(...) { ... }  
    void chargeForPremium(...) { ... } // billing?  
    void resizeRoomPhoto(...) { ... } // images?  
    String formatDate(...) { ... }    // ...sure, why not  
}
```

Four reasons to change, four audiences, one class. Split it along its responsibilities.

How a class gets that way



programmerhumor.io

Don't talk to strangers

```
order.getCustomer().getAddress().getCity().getName();
```

- Every `.get()` couples us to one more class's structure.
- Break any link in that chain and our code breaks too.
- **The Law of Demeter:** Don't reach through objects to get at their friends.
- **Tell, don't ask:** The same idea from the caller's side. Ask the object to do the thing instead of pulling its data out and deciding for it.

Let's practice: critique this design

```
class Shipment {  
    private List<Box> boxes;  
    int getWeight() {  
        int w = 0;  
        for (Box box : boxes)  
            for (Item item : box.getItems())  
                w += item.weight;  
        return w;  
    }  
}  
  
class Box {  
    private List<Item> items;  
    Iterable<Item> getItems() { return items; }  
}  
  
class Item { String description; int weight; }
```



Canvas → Quizzes → Lecture 7 In-Class Quiz

Finding this in a codebase you did not write

- Name the problem and ask the agent where it might be. "Where does this code reach through two getters?" Then verify the hits yourself.
- Tools find shapes. A linter's design rules, an import graph, a `git log` of files that change together.
- Start at the file your change touches and follow its imports outward. Do not audit the whole repo.
- Some "smells" are greppable. `Manager`, `Util`, a getter chain with two dots, a long parameter list.

```
$ pmd check -R category/java/design.xml
```

```
RequestHandler.java:12 UseObjectForClearerAPI  
  Rather than using a lot of String  
  arguments, consider using a container  
  object for those values.
```

```
RequestHandler.java:45 UseObjectForClearerAPI  
  (same message)
```

```
RequestHandler.java:67 UseObjectForClearerAPI  
  (same message)
```

```
[INFO] Found 3 violations.
```

PMD 7.27.0, Java design rules, run on the Lab 3 starter. Paths shortened and messages wrapped to fit.

Symptoms

Too much coupling

- **Shotgun surgery:** One small change forces edits across a dozen files.
- **Ripple effects:** A fix here breaks something unrelated over there.
- You can't test one piece without dragging half the system along.

Weak cohesion

- A class named `Manager`, `Util`, or `Helper` that quietly does everything.
- Methods that never touch the class's own fields (squatting in the wrong house).
- "What is this class **for**?" takes a whole page to answer.

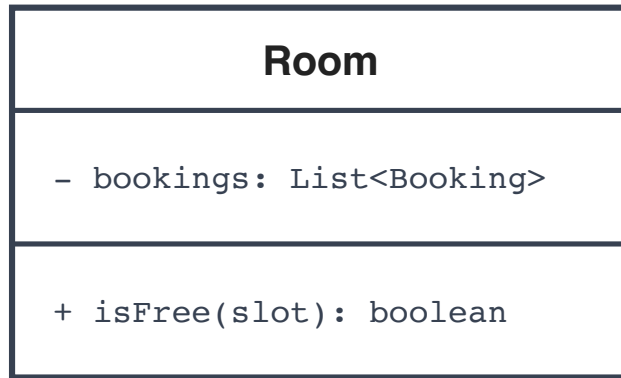
Where does behavior live?

Recall "a room can't be double-booked".

**Where does each responsibility
go?**

Information Expert

- Assign a responsibility to the class that **has the information** to fulfill it.
- Who decides if a room is free at a given time? The thing that holds the room's bookings.
- We made this argument last week ("the room knows its bookings").
- Behavior wants to live next to the data it needs.



the data

the check

same box, on purpose

behavior lives next to the data it needs

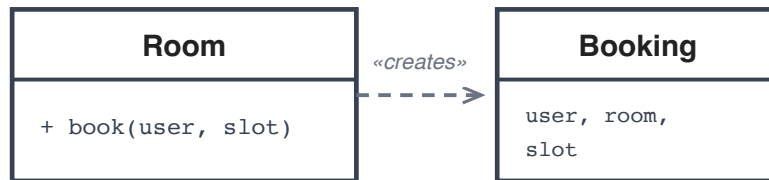
Information Expert in code

```
class Room {  
  private List<Booking> bookings;  
  boolean isFree(Slot s) {                // the data lives here...  
    return bookings.stream()  
      .noneMatch(b -> b.overlaps(s));    // ...so the check does too  
  }  
}
```

Put `isFree` anywhere else and you'd have to hand out the bookings so someone else can scan them. Don't.

Creator

- Who creates a **Booking**? The **Room** that contains it and has the data to initialize it.
- Assign creation to the class that aggregates, contains, or closely uses the new object.
- It usually already has what the constructor needs.

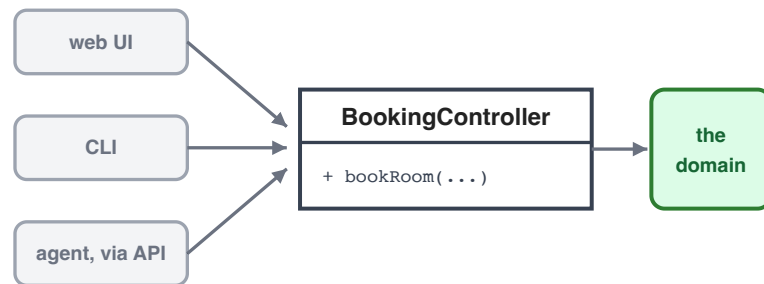


the container creates its contents

it already holds what the constructor needs

Controller

- Who handles a system operation like "book a room"? A dedicated controller, not the UI widget that was clicked.
- Keeps the rules out of the buttons, so the same logic serves web, CLI, and an agent calling an API.



one front door, three kinds of visitor

the rules live past the button, so every caller gets the same ones

GRASP = General Responsibility Assignment Software Patterns. Heuristics, not laws.

One controller, or one per use case?

Façade controller

- One controller for the whole system:
`ReservationSystem.book(...)`,
`.cancel(...)`,
`.listBookings(...)`.
- Fine while the system has a handful of operations.
- It receives every event, so it can bloat into a `Manager` class.

Use-case controller

- One per use case: `BookRoom`,
`CancelBooking`,
`PromoteFromWaitlist`.
- Each stays small and changes for one reason.
- More classes to find your way around. Worth it once the operations are many.

Either way, the UI and the domain depend only on the controller.

Two designs for the waitlist

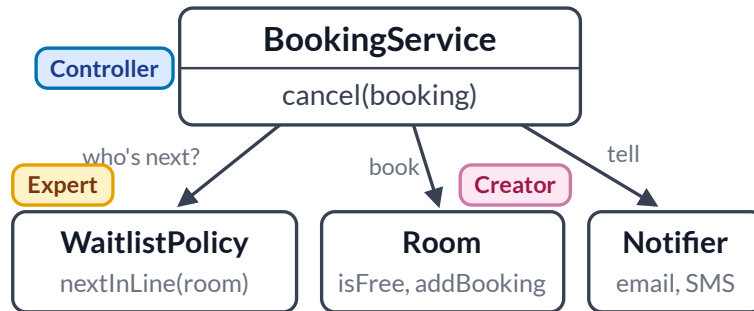
A booking is cancelled. Someone on the waitlist should get the slot. Two candidate designs:

A: Room does it all



Room now changes for three reasons:
booking rules, waitlist policy, notifications.

B: A coordinator

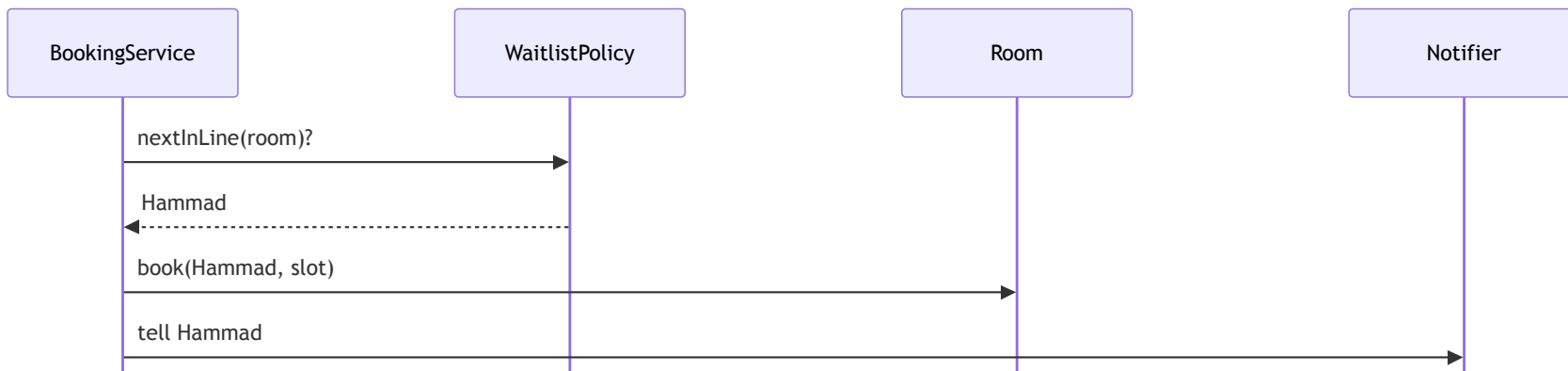


Each box knows one thing.
Only the service knows all three.

Coupling: who knows whom? Cohesion: one reason to change?

Who promotes from the waitlist?

- **Room** ? It knows its bookings, but not the waitlist, or how to notify anyone.
- **WaitlistPolicy** ? It knows **who's next**. That's all it knows.
- **BookingService** ? It coordinates. Asks the policy who, books them into the room, has the notifier tell them.
- The rule spans several experts, so a coordinator calls them. Controller and Information Expert cooperating.



Which design survives which change?

Three changes from the list earlier. Which classes does each one touch?

Change	A: Room does it all	B: A coordinator
Notify by SMS as well	Inside <code>Room.notify()</code> , next to the booking rules	<code>Notifier</code> only
Pricing tiers	More rules inside <code>Room</code>	A new policy the service asks, like the waitlist one
Recurring bookings	<code>book()</code> , <code>cancel()</code> , and the waitlist logic all move	<code>book()</code> in the service and <code>Room</code> 's bookings. The policies stay.

(You are arguing about **modifiability**. Say which change, not just "it is cleaner to change".)

Names for things you already did

- **Protected Variations:** Put a stable interface around the thing that will change. That was "hide what's likely to change".
- **Pure Fabrication:** Invent a class the domain does not have, because the responsibility fits nowhere else. `BookingService` and `WaitlistPolicy` are both fabrications.
 - What does this do to our representational gap?
- **Indirection:** Route through a middle object so two things need not know each other. Your adapter between the framework and the domain.
- Low Coupling and High Cohesion are GRASP names as well.

A quick debrief from Lab 3

What the starter had

The overlap rule lived in the request handler, as an entry-point rule. A second, different check lived in the store.

The class the design doc described, `BookingPolicy`, was complete and never called.

Storage was a concrete class reached directly. Tests could not substitute it, and a storage swap would have edited the handler.

Connecting it to design goals/heuristics/principles

One home for the rule, next to the bookings it checks (Information Expert). Every entry point calls it, so no path can skip it.

The rule's home already existed, so the fix was calling it from every entry point.

The store behind an interface (hide what's likely to change). A test can substitute it, and a swap touches one class.

In the age of coding agents

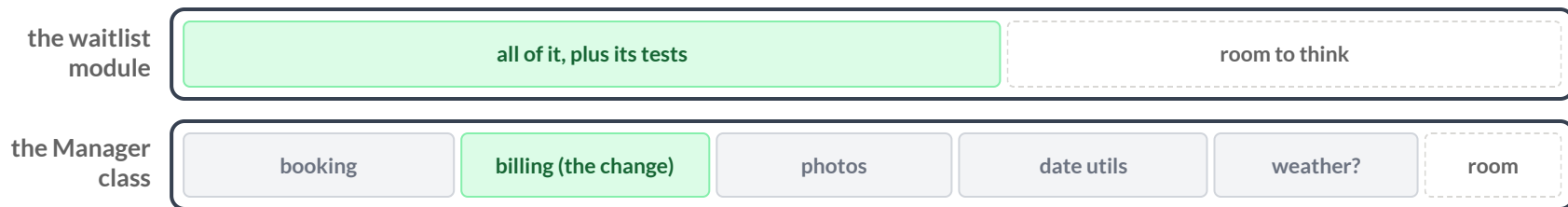
Coupling = agent's reach

- Low coupling means a module an agent can change without touching the neighbors.
- Tight coupling means one "small" generated change quietly breaks other things.
 - The agent may not notice. Each edit into a neighbor looks reasonable on its own, and the file count in the diff shows the damage.
- The looser the coupling, the safer the delegation.

Cohesion = "fits in one focused context"

Think of modularity as the unit of safe delegation (and the unit of human review).

- A highly cohesive module is a task that fits in one focused context.
- The agent reads what it needs and nothing else, and so do you, in review.
 - *Your* reading budget for a change: read the interface it touches, the check that guards it, the diff summary, and the tests. The suite covers the rest.
- A sprawling low-cohesion class fits in nobody's head, human or model. The window is big now, and what it is full of still matters.



X ...it read all five to change one, so its edit can reach all five

Decompose along the boundaries

- A task in your plan should name the module it lives in and the interface it may touch.
- A small task that touches three modules is a smell in the plan before it is a smell in the diff.
- Boundaries decide what an agent can be handed. Settle them first, then cut the tasks.

A plan cut along the boundaries

New requirement: **a full room can now have a waitlist.**

Task 1: domain/WaitlistPolicy

A FIFO of requests per room. Touches nothing else.

Interface: add, nextInLine(room), remove.

Check: ordering and removal, unit tests on the class alone.

Task 2: domain/BookingService.cancel

After removing the booking, ask WaitlistPolicy.nextInLine(room) and book it. Crosses one boundary, the WaitlistPolicy interface.

Check: empty waitlist, cancel behaves as before.

One entry, it is promoted.

Task 3: api, the room view

Show the waitlist. Read-only. No rule lives here.

Check: the response shape.

For A2

We care about the planning discipline, not the format.

Three shapes that work:

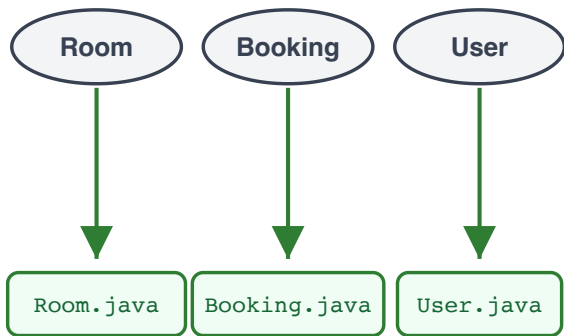
- per feature, like this
- per harness check, thinnest path first
- per layer (store, then domain, then API)

Whichever you pick, the agent can draft it. You cut, split, and add the checks.

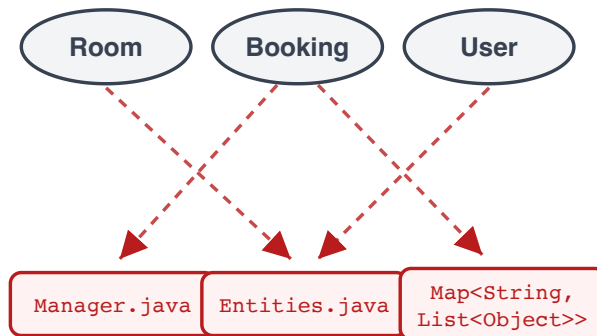
(Task 2 is the only one that crosses a boundary, and it says which.)

Back to the representational gap

- If the design mirrors the domain, the agent stays oriented while changing it.
- `room.isFree(slot)` is obvious. What is `svc.process(data, 2, true)`?
- Clean responsibilities generally keep the agent's changes where they belong.



low gap: read the code, recognize the problem



high gap: every reader translates in their head

The same change in two designs

In the well-designed system

Consider the waitlist requirement in a design that was ready for it:

- **High cohesion:** The booking logic is in one place, so there's one place to extend.
- **Low coupling:** Billing, room photos, and auth never knew how booking worked, so they don't care.
- The change stays local.

The diff:

```
domain/WaitlistPolicy.java +82
domain/BookingService.java +9
```

In the badly designed system

- Booking logic is smeared across the UI, the store, and three helpers.
- "Add a waitlist" now touches all of them. Billing and auth need re-checking too, because everything reaches into everything.
- Same feature, far more work, and new bugs.

The diff:

```
ui/BookingForm.js      +31
ui/AdminPanel.js       +12
svc/BookingHelper.java +44
svc/Utils.java         +17
store/procedures.sql   +28
billing/Recheck.java   +6   ...and 4 more
```

You already have a change coming

Note that the API itself will also change in future assignments. We are not telling you how, on purpose. A revision will not remove endpoints, change response shapes, or repurpose existing fields, but it may add status codes to existing calls or change what an existing call means.

- your A2 contract

- The core paths are the stable thing. Your additions go under `x_` fields and `/api/x/` routes, so a revision cannot collide with them.
- That is Protected Variations written into the contract for you.

But how modular should it be?

It depends.

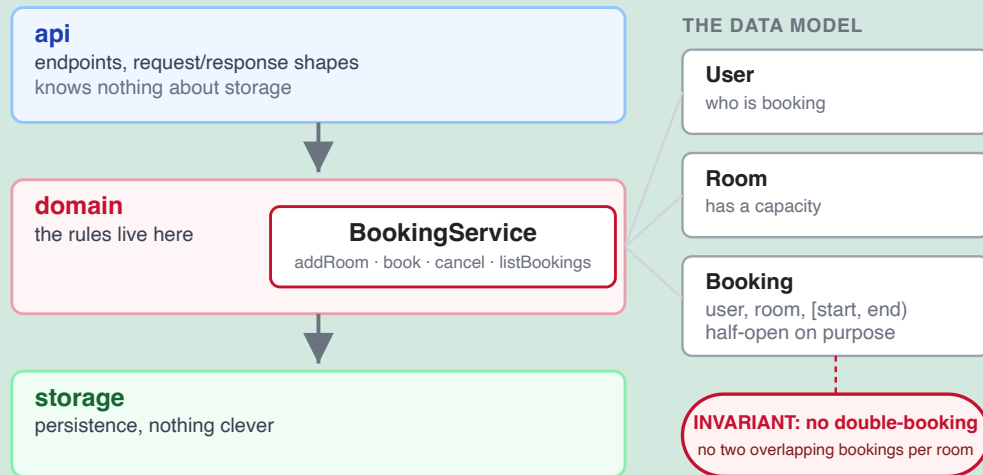
On what?

- How likely the change is. A boundary for a change that never comes is complexity you pay for now and may never use.
- Who owns the callers. If you own them all, you can change the interface later. If you don't, it has to be right now.
- Whether you can name a second thing that would sit behind the interface (email and SMS, in-memory and Postgres). With only one, its shape is a guess.
- The extremes: everything in one class (no coupling, no cohesion), or every statement its own module (perfect cohesion, hopeless coupling).

Let's practice: what else breaks?

Our layered booking system has storage, domain, and api layers, with the waitlist in domain. For each change, name every module it touches:

- Swap the in-memory store for Postgres.
- Rooms get per-hour pricing.
- Bookings now need approval from a room owner.
- The UI adds a calendar view.



Which change is this design **not** ready for, and what boundary would you add?

Summary

- Change is guaranteed. Design to make the **likely** changes cheap and local.
- Aim for low coupling, high cohesion. Talk through interfaces, not internals.
- Use GRASP heuristics (Information Expert, Creator, Controller) to decide where behavior lives. Apply them to the diagram before the code exists.
- For agents: coupling is reach, cohesion is one focused context. **Modularity is the unit of safe delegation.**
- Decouple what varies. Over-decoupling is its own cost. (How much? It depends.)

What's next

W1 A1 out	W2	W3 A1 due	W4 A2 chkpt	W5 A2 due · M1	W6	W7 A3 due	W8	W9 A4 due	W10 M2	W11	W12 A5 due	W13	W14 A6 due
---------------------	-----------	---------------------	-----------------------	--------------------------	-----------	---------------------	-----------	---------------------	------------------	------------	----------------------	------------	----------------------

- **Friday, Lab 4:** First AWS deploy.
- **Monday:** Modularity gone wrong. Anti-patterns, including the kinds agents are more likely to produce. And A2 is due that night, 11:59pm.
- **Next Wednesday:** Midterm 1.